

Estado	Finalizado
Comenzado	viernes, 24 de julio de 2026, 17:54
Completado	viernes, 24 de julio de 2026, 19:10
Duración	1 hora 16 minutos
Calificación	100 de 100
Comentario -	Aprobado

Pregunta 1

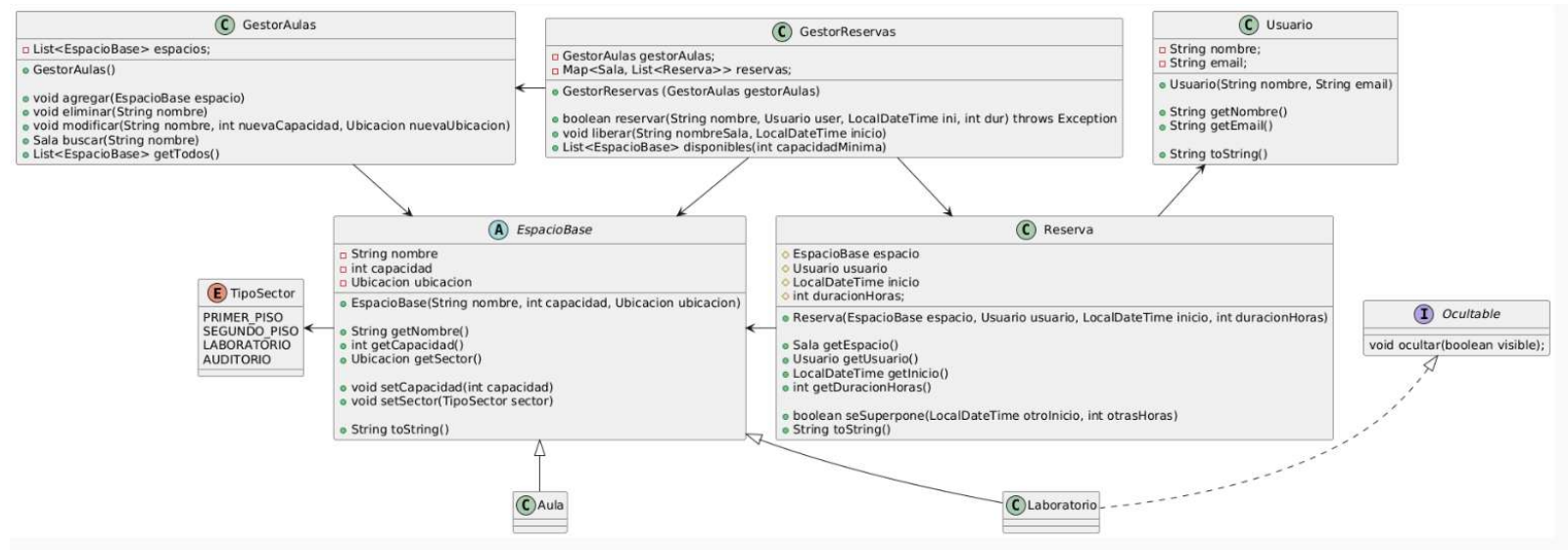
Finalizado

Sin calificar

Se provee un sistema parcial para la gestión de reservas de Espacio. El sistema permite registrar Espacios (Aulas, laboratorios, etc) con distintas capacidades y ubicaciones, asignar reservas a usuarios por varias horas, listar salas disponibles y cancelar reservas.

La tarea consiste en **completar e implementar las partes faltantes** del sistema según las consignas que se detallan a continuación. El objetivo es evaluar tu dominio de:

- Creación y uso de clases, objetos y colecciones.
- Implementación de constructores, getters, setters y `toString`.
- Uso de `enum`, herencia e interfaces.
- Manejo de excepciones.
- Comprensión del diseño orientado a objetos.

Descargar proyecto

Examen de Infomratica Avanzada. Segunda mesa de julio 2026. Alumna: Juana Pucheta Noguera.

Pregunta 2

Correcta

Se puntúa 15 sobre 15

Completar la clase **EspacioBase**:

- Implementar el constructor.
- Definir los métodos **getNombre()**, **getCapacidad()** y **getSector()**.
- Implementar los métodos **setCapacidad(...)** y **setSector(...)**.
- Sobrescribir el método **toString()** para mostrar una descripción legible de la sala.

Objetivos evaluados: creación de objetos, constructores, getters/setters, sobreescritura de métodos.**Nota:** Esta es una clase **abstracta**, para su validación asumimos la existencia de la clase **Aula** que extiende de **EspacioBase****Por ejemplo:**

Prueba	Resultado
<pre>EspacioBase s = new Aula("Aula107", 6, TipoSector.PRIMER_PISO); System.out.println (s.getNombre()); System.out.println (s.getSector()); System.out.println (s.getCapacidad()); System.out.println (s);</pre>	<pre>Aula107 primer piso 6 Aula107 (6 personas, primer piso)</pre>

Respuesta: (sistema de penalización: 0, 0, 0, 5, 10, 15 %)

Reiniciar respuesta

```
1 public abstract class EspacioBase {
2     protected String nombre;
3     protected int capacidad;
4     protected TipoSector sector;
5
6     /**
```

```
7      * Constructor con parámetros
8      * Inicializa las variables de instancia
9      */
10 public EspacioBase(String nombre, int capacidad, TipoSector sector) {
11     this.nombre = nombre;
12     this.capacidad = capacidad;
13     this.sector = sector;
14 }
15
16 public String getNombre() {
17     return nombre;
18 }
19
20 public int getCapacidad() {
21     return capacidad;
22 }
23
24 public TipoSector getSector() {
25     return sector;
26 }
27
28 /**
29  * Setea la capacidad de la sala
30  * solo si la capacidad es mayor a 0
31  */
32 public void setCapacidad(int capacidad) {
33     if(capacidad > 0){
34         this.capacidad = capacidad;
35     }
36 }
37
38 public void setSector(TipoSector sector) {
39     this.sector = sector;
40 }
41
42 @Override
43 /**
44  * Retorna una representación del Espacio Base como String con el formato
45  *
46  * "<nombre> (<capacidad> personas, <sector>)"
47  *
48  * Ejemplo:
```

```
49      *      lab1 (6 personas, Piso dos)
50      */
51      public String toString() {
52          return this.nombre + " (" + this.capacidad + " personas. " + this.sector + ")";
```

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 15/15.

Pregunta 3

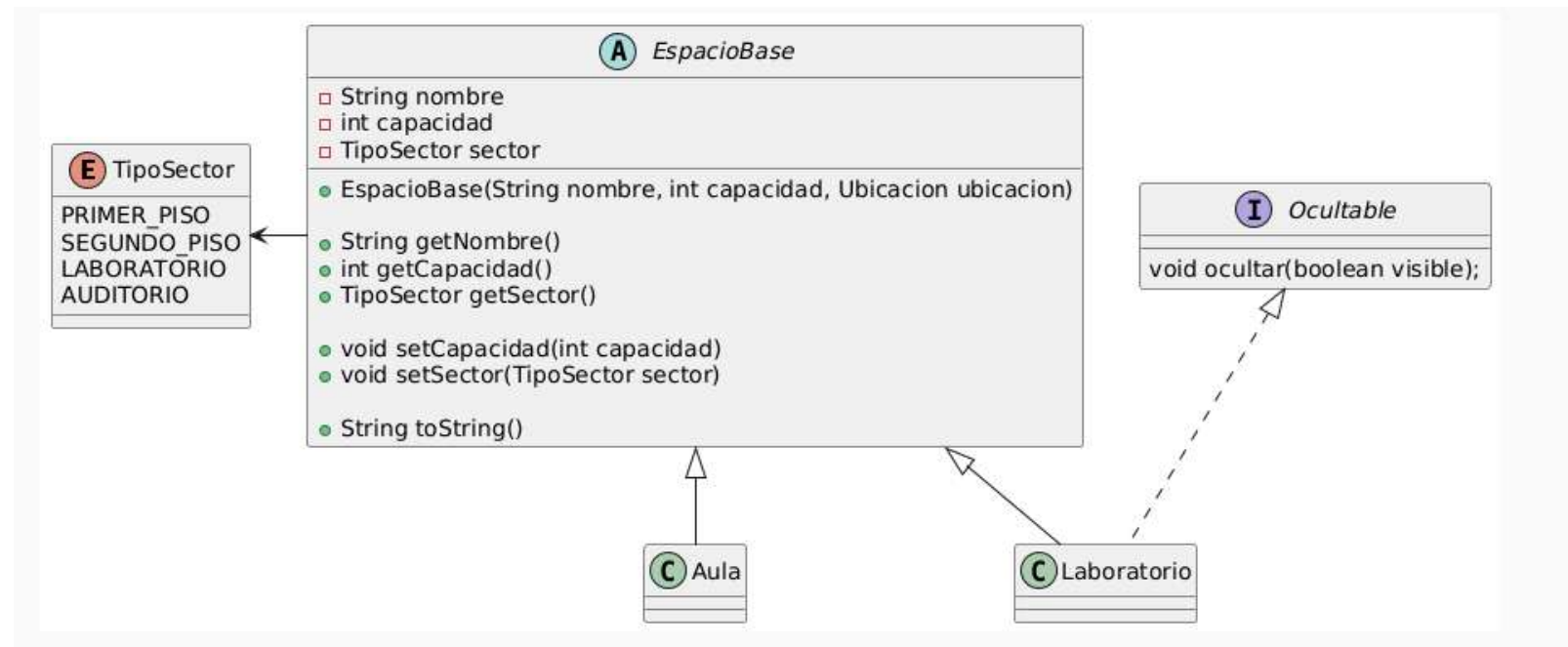
Correcta

Se puntúa 15 sobre 15

Implemente la Clase **Laboratorio** tal que

- Extienda de **EspacioBase**
- Implemente la interfaz **Ocultable**
- El método `ocultar(boolean visible)` muestre un mensaje simple por consola
 - "**Laboratorio <nombre> [visible|oculto] para el sistema de reservas.**"
 - Ej. "*Laboratorio LAB1 oculto para el sistema de reservas.*"

Objetivos evaluados: herencia, uso de **super**, interfaces, polimorfismo.



Por ejemplo:

Prueba	Resultado
<pre> Laboratorio l = new Laboratorio("Lab-1", 5, TipoSector.PRIMER_PISO); System.out.println ("Hereda de EspacioBase: " + (l instanceof EspacioBase)); System.out.println ("Hereda de Ocultable: " + (l instanceof Ocultable)); System.out.println (l); l.ocultar(true); l.ocultar(false); </pre>	<pre> Hereda de EspacioBase: true Hereda de Ocultable: true Lab-1 (5 personas, primer piso) Laboratorio Lab-1 visible para el sistema de reservas. Laboratorio Lab-1 oculto para el sistema de reservas. </pre>

Respuesta: (sistema de penalización: 0, 0, 0, 5, 10, 15 %)

Reiniciar respuesta

```

1 public class Laboratorio extends EspacioBase implements Ocultable{
2
3     // Constructor
4     public Laboratorio (String nombre, int capacidad, TipoSector sector){
5         super(nombre, capacidad, sector);
6     }
7
8     // Muestro es mensaje segun si la visibilidad esta oculta o visible.
9     public void ocultar(boolean visible){
10        // Escribo la forma corta de if / else aprovecho que son mensajes cortos.
11        String estado = visible ? "visible" : "oculto";
12        System.out.println("Laboratorio " + this.getNombre() + " " + estado + " para el sistema de reservas.");
13    }
14 }

```


Todas las pruebas superadas. 

Correcta

Puntos para este envío: 15/15.



Pregunta 4

Correcta

Se puntúa 20 sobre 20

Completa los métodos indicados de la clase **Reserva** según la documentación

- Implementar el método **toString** con formato: "Reserva de <nombre_aula> por <usuario> desde <fecha_hora_inicio> por <duracion>h".
- Implementar el método **seSuperpone(...)** para verificar si una reserva se superpone con otra en función del horario.

El sistema utiliza la clase **LocalDateTime** de la biblioteca **java.time** para representar fechas y horas. Esta clase permite expresar momentos como "2025-06-15T10:00" y sumar horas usando el método **plusHours(int)**. También pueden compararse con operadores **>**, **<**, o utilizando el método **isBefore(LocalDateTime)**. No es necesario considerar zonas horarias ni diferencias geográficas. Todos los horarios se interpretan como locales.

Objetivos evaluados: uso de **LocalDateTime**, lógica condicional, comprensión de intervalos.

Por ejemplo:

Prueba	Resultado
<pre>Aula a = new Aula ("Aula-106",40,TipoSector.PRIMER_PISO); Usuario u = new Usuario ("Juan Perez", "jp@mail.com"); LocalDateTime ldt = LocalDateTime.parse("2025-06-16T10:00:00"); Reserva r = new Reserva(a,u,ldt,1); System.out.println (r); System.out.println ("9am, 1 hora se superpone: "+ r.seSuperpone(LocalDateTime.parse("2025-06-16T09:00:00"),1)); System.out.println ("9am, 2 horas se superpone: "+ r.seSuperpone(LocalDateTime.parse("2025-06-16T09:00:00"),2));</pre>	<pre>Reserva de Aula-106 por Juan Perez <jp@mail.com> desde 2025-06-16T10:00 por 1h 9am, 1 hora se superpone: false 9am, 2 horas se superpone: true</pre>

Respuesta: (sistema de penalización: 0, 0, 0, 5, 10, 15 %)

[Reiniciar respuesta](#)

```
4     protected EspacioBase espacio;
5     protected Usuario usuario;
6     protected LocalDateTime inicio;
7     protected int duracionHoras;
8
9     public Reserva(EspacioBase espacio, Usuario usuario, LocalDateTime inicio, int duracionHor
10         this.espacio = espacio;
11         this.usuario = usuario;
12         this.inicio = inicio;
13         this.duracionHoras = duracionHoras;
14     }
15
16     public EspacioBase getEspacio() { return espacio; }
17     public Usuario getUsuario() { return usuario; }
18     public LocalDateTime getInicio() { return inicio; }
19     public int getDuracionHoras() { return duracionHoras; }
20
21     /**
22      * indica si el horario de esta Reserva se superpone con otro horario y duración
23      * pasada como parámetro
24      *
25      * Por ejemplo, si esta reserva es de 9 a 11,
26      * otroInicio 10, otrasHoras 1 -> true
27      * otroInicio 8, otrasHoras 2 -> True
28      * otroInicio 8, otrasHoras 1 -> false
29      *
30      */
31     public boolean seSuperpone(LocalDateTime otroInicio, int otrasHoras) {
32         LocalDateTime fin = this.inicio.plusHours(this.duracionHoras);
33         LocalDateTime otroFin = otroInicio.plusHours(otrasHoras);
34
35         // Si tengo el primer rango [A, B] y otro rango con [C, D], se van a superponer si
36         // sucede que (A < D) y (C < B). Comparo eso entonces.
37
38         return this.inicio.isBefore(otroFin) && otroInicio.isBefore(fin);
39     }
40
41     @Override
42     /**
43      * Retorna un string que representa la Reserva con el formato
```

```
43 ~ retorna un string que represente la reserva con el formato
44 * "Reserva de <nombre_sala> por <usuario> desde <hora_inicio> por <duracion> horas"
45 * Ej
46 * Reserva de Uritorco por Juan Perez <jp@mail.com> desde 2025-06-16T10:00 por 1h"
47 */
48 public String toString() {
49     String info = "Reserva de " + this.espacio.getNombre() +
50                 " por " + this.usuario.toString() +
51                 " desde " + this.inicio.toString() +
52                 " por " + this.duracionHoras + "h" ;
53     return info;
54 }
55 }
```

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 20/20.

Pregunta 5

Correcta

Se puntúa 25 sobre 25

Completar los métodos de **GestorAulas** para:

- Agregar espacios (agregar).
- Eliminar espacios por nombre (eliminar).
- Modificar capacidad y ubicación (modificar).
- Buscar un espacio por nombre (buscar).
- Obtener la lista completa (getTodos).

Objetivos evaluados: uso de colecciones (List), búsqueda, eliminación y modificación de elementos.

Por ejemplo:

Prueba	Resultado
<pre>GestorAulas gestorAulas = new GestorAulas(); Aula a1 = new Aula ("Verde", 10, TipoSector.PRIMER_PISO); Aula a2 = new Aula ("Azul", 20, TipoSector.SEGUNDO_PISO); gestorAulas.agregar(a1); gestorAulas.agregar(a2); System.out.println (gestorAulas.getTodos());</pre>	[Verde (10 personas, primer piso), Azul (20 personas, segundo piso)]

Respuesta: (sistema de penalización: 0, 0, 0, 5, 10, 15 %)

Reiniciar respuesta

```
1 import java.util.*;
2
3 public class GestorAulas {
4     private List<EspacioBase> espacios = new ArrayList<>();
5 }
```

```
~
6 ▾ /**
7   * Agrega un Espacio a la lista de Espacios del gestor
8   */
9 ▾ public void agregar(EspacioBase espacio) {
10 ▾     if(espacio != null){
11         this.espacios.add(espacio);
12     }
13 }
14
15 ▾ /**
16   * Elimina de la lista el EspacioBase cuyo nombre coincide
17   * con el nombre pasado como argumento.
18   */
19 ▾ public void eliminar(String nombre) {
20     EspacioBase encontrado = buscar(nombre);
21
22     if(encontrado != null){
23         this.espacios.remove(encontrado);
24     }
25 }
26
27 ▾ /**
28   * Actualiza la capacidad y ubicación del EspacioBase cuyo
29   * nombre coincide con el nombre pasado como argumento.
30   */
31 ▾ public void modificar(String nombre, int nuevaCapacidad, TipoSector nuevoSector) {
32     EspacioBase encontrado = buscar(nombre);
33
34     if(encontrado != null){
35         encontrado.setCapacidad(nuevaCapacidad);
36         encontrado.setSector(nuevoSector);
37         // encontrado.setUbicacion(nuevoSector);
38     }
39 }
40
41 ▾ /**
42   * Devuelve el EspacioBase cuyo nombre coincide con el nombre
43   * pasado como argumento
44   */
45 ▾ public EspacioBase buscar(String nombre) {
46 ▾     for(EspacioBase espacio : this.espacios){
```

```
47 |         if (espacio.getNombre().equals(nombre)){  
48 |             return espacio;  
49 |         }  
50 |     }  
51 |     return null;  
52 | }
```

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 25/25.

Pregunta 6

Correcta

Se puntúa 25 sobre 25

La clase **GestorReservas** administra las reservas de Espacios. Posee un **GestorAulas** que gestiona los EspaciosBases, y un **Mapa** que relaciona cada **EspacioBase** con una Lista de Reservas.

Hay que implementar los siguientes métodos.

- Agregar una reserva a un Espacio(reservar).
- Libera una reserva de una Espacio (liberar).
- Lista todas los Espacios disponibles (disponibles).

Nota: puede utilizar los métodos **seSuperpone** de la clase Reserva para determinar si un horario y duración se superpone con la reserva.

Objetivos evaluados: uso de colecciones (List, Map), búsqueda, eliminación y modificación de elementos.

Por ejemplo:

Prueba	Resultado
<pre>// Setup del TEST: Crea gestores, salas, usuarios, y hora GestorAulas gestorAulas = new GestorAulas(); GestorReservas gestorReservas = new GestorReservas(gestorAulas); Aula a1 = new Aula ("Verde", 10, TipoSector.PRIMER_PISO); gestorAulas.agregar(a1); Usuario ana = new Usuario("Ana", "ana@mail.com"); LocalDateTime ahora = LocalDateTime.of(2025, 6, 15, 10, 0); // Test Reservar System.out.println("Reservando ..."); try { gestorReservas.reservar("Verde", ana, ahora, 2); } catch (Exception e) { System.out.println("Error al reservar: " + e.getMessage()); } // Verifica las reservas System.out.println("\n\nReservas"); for (Reserva r : gestorReservas.getReservasPorEspacio("Verde")){ System.out.println(r); } // Test disponibles System.out.println("\n\nEspacios disponibles para una reunion de 1hr a las "+ahora); for (EspacioBase s : gestorReservas.disponibles(ahora, 1)) { System.out.println(s); } System.out.println("\n\nEspacios disponibles por 1hr a las "+ahora.plusHours(2)); for (EspacioBase s : gestorReservas.disponibles(ahora.plusHours(2), 1)) { System.out.println(s); } // Test liberar Sala</pre>	Reservando ... Reservas Reserva de Verde por Ana <ana@mail.com> desde 2025-06-15T10:00 por 2h Espacios disponibles para una reunion de 1hr a las 2025-06-15T10:00 Espacios disponibles por 1hr a las 2025-06-15T12:00 Verde (10 personas, primer piso) Liberando espacio Verde a las 2025-06-15T10:00 Reservas espacio Verde:

Prueba	Resultado
<pre>System.out.println("\n\nLiberando espacio Verde a las " + ahora); gestorReservas.liberar("Verde", ahora); // Verifica reservas System.out.println("\nReservas espacio Verde:"); for (Reserva r : gestorReservas.getReservasPorEspacio("Verde")) { System.out.println(r); }</pre>	

Respuesta: (sistema de penalización: 0, 0, 0, 5, 10, 15 %)

Reiniciar respuesta

```
4 import java.util.List;
5 import java.util.Map;
6
7 class GestorReservas {
8     private GestorAulas gestorAulas;
9     protected Map<EspacioBase, List<Reserva>> reservas;
10
11     public GestorReservas(GestorAulas gestorAulas ) {
12         this.gestorAulas = gestorAulas;
13         this.reservas = new HashMap<EspacioBase, List<Reserva>>();
14         for (EspacioBase e : gestorAulas.getTodos()) {
15             reservas.put(e, new ArrayList<>());
16         }
17     }
18
19
20     public List<Reserva> getReservasPorEspacio (String nombreEspacio){
21         EspacioBase espacio = gestorAulas.buscar(nombreEspacio);
22         List<Reserva> listaReservas=null;
23         if (espacio !=null && reservas.containsKey(espacio)) {
24             listaReservas = reservas.get(espacio);
25         }
26         return listaReservas;
27     }
28 }
```

```
29
30 ▾ /**
31     * Reserva un espacio para un usuario, a la hora y con la duración indicada
32     *
33     * - Si el espacio no existe en el gestor -> Lanza Exception ()
34     * - Revisa todas las reservas asociadas al espacio, y si existe alguna
35     *   que se superpone -> Lanza Exception ()
36     *
37     * - Si el espacio existe, y no tienen ninguna reserva superpuesta:
38     *     - se genera una nueva Reserva
39     *     - se agrega al mapa de reservas
40     *     - retorna true
41     *
42     * @throw Exception si la sala no existe, o si tiene una reserva en el
43     * horario requerido
44     * @return true si se pudo generar la reserva.
45     *
46     */
47 ▾ public boolean reservar(String nombreEspacio, Usuario usuario, LocalDateTime inicio, int d
48     EspacioBase espacio = gestorAulas.buscar(nombreEspacio);
49
50 ▾     if(espacio == null){
51         throw new Exception();
52     }
53
54 ▾     if(!reservas.containsKey(espacio)){
55         reservas.put(espacio, new ArrayList<>());
```

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 25/25.

